

■■■ CallShield AI

College Review & Viva Presentation Guide (Complete Reference)

1. Executive Summary (Project Pitch)

CallShield AI is an intelligent real-time scam call detection and protection platform designed for Android smartphones and web environments. It automatically intercepts active incoming and outgoing phone calls, converts spoken speech into text using continuous speech recognition, and applies Machine Learning (TF-IDF Feature Extraction + Calibrated Classification) to compute fraud risk scores (0–100%) and alert users to scam threats in real time.

2. Technology Stack

Layer	Technology Used	Purpose / Responsibility
Android App	Java, Native XML Layouts	Builds native dialer, contact list, overlay UI, and background phone state receiver.
Web Frontend	HTML5, TailwindCSS, FontAwesome, JS	Renders real-time risk meter, speaker transcript bubbles, and dashboard controls.
Backend API	Python 3.10+, Flask	Serves REST API endpoints (/api/analyze_live) to process text and calculate risk metrics.
Machine Learning	Scikit-Learn, TF-IDF Vectorizer	Extracts textual n-grams and computes calibrated fraud probability percentage.
Speech Recognition	Android SpeechRecognizer & WebSpeech	Converts live spoken audio to text continuously in Hindi (hi-IN) & English.
Hosting & Cloud	Vercel & GitHub	Provides live cloud deployment and version control.

3. Website & ML Backend Files (Scam_audio_Full_Project)

File Path	Key Function & Responsibility
app.py	Main Flask application server. Handles routing, loads ML models, and serves /api/analyze_live endpoint.
models/scam_detector_calibrated.pkl	Trained Scikit-Learn classifier model file that outputs fraud risk probability (0% to 100%).
models/vectorizer.pkl	TF-IDF Vectorizer file that transforms raw speech text into numerical feature matrices.

templates/modules/live_call.html	Live call detection UI. Displays the animated circular risk gauge, speaker transcript, and red flag badges.
templates/dashboard.html	System dashboard displaying analytics, system status, and links to recorded call logs.
generate_research_pdf.py	Automated report generator script producing official academic documentation.

4. Android App Architecture Files (CallShield_Android_App)

File Path	Key Function & Responsibility
MainActivity.java	Main Activity controller. Controls native dialpad, native contacts list, permissions, and overlay container.
CallReceiver.java	BroadcastReceiver intercepting PHONE_STATE_CHANGED and NEW_OUTGOING_CALL when a call connects.
CallShieldService.java	Foreground Service with high-priority notification to maintain live call scanning without background kill.
activity_main.xml	Native layout defining sleek dark keypad buttons, tab switcher, and hidden overlay view container.
item_contact.xml	Custom layout for contact items featuring colored initial avatars, name, phone number, and call button.
AndroidManifest.xml	Declares system permissions (RECORD_AUDIO, READ_CALL_LOG, SYSTEM_ALERT_WINDOW, FOREGROUND_SERVICE).

5. End-to-End System Workflow (Step-by-Step)

- Step 1 (Call Connect):** User dials a number or receives an incoming call.
- Step 2 (State Interception):** Android *CallReceiver* & *CallShieldService* detect CALL_STATE_OFFHOOK.
- Step 3 (Overlay Display):** *MainActivity* brings the zero-click AI scanner overlay to the screen.
- Step 4 (Speech to Text):** Audio is captured and converted to transcript text continuously.
- Step 5 (ML Backend Request):** Transcript text is sent via HTTP POST to /api/analyze_live.
- Step 6 (Feature Extraction & Scoring):** TF-IDF vectorizer extracts n-gram patterns (e.g., 'bank OTP', 'compromised account'), and the Calibrated Classifier calculates the fraud risk score (%).
- Step 7 (Real-Time Alert):** The UI updates the circular risk meter dynamically (Green = Safe, Red = Scam Confirmed).

6. Anticipated Review Questions & Ideal Answers

Q1: Why did you use TF-IDF + Calibrated Classifier instead of simple keyword matching?

Answer: Keyword matching is rigid and easily circumvented by minor word variations. TF-IDF measures word frequency and rarity across datasets, allowing our Machine Learning model to evaluate contextual n-grams and

generate a calibrated probability percentage (e.g. 88.5% risk) rather than binary matches.

Q2: How does the Android app intercept calls automatically without clicking a button?

Answer: We implemented an Android *BroadcastReceiver* (CallReceiver) paired with a *ForegroundService* (CallShieldService) and requested *SYSTEM_ALERT_WINDOW* ('Display over other apps') permission. When the telephony state transitions to OFFHOOK, Android permits our service to launch the scanner activity automatically.

Q3: What languages are supported by the live scanner?

Answer: The speech engine supports Hindi (hi-IN), Marathi (mr-IN), Indian English (en-IN), and US English (en-US), making it tailored for diverse real-world Indian fraud attempts.